altairsim — Changelog

2026-08-19 · 31e0504

A simulation of the MITS Altair 8800 and the S-100 bus

Changelog

Every release of **altairsim**, newest first. Each entry is the short version — what you can do here that you could not do in the release before it. The User Manual describes the program as it is now; this document is the record of how it got there.

0.4.0

0.4.0 is the boards release. New S-100 boards join the backplane, and five new machine families (Cromemco, SD Systems, Tarbell, iCOM, S100Computers) boot alongside the MITS originals for the first time — disks and cassettes learn to be built and formatted by the guest instead of only read, and the debugger, the monitor prompt and the video window all got noticeably sharper along the way.

Five new machine families join the MITS originals

Cromemco arrives as a full boot chain: the **16FDC/64FDC** floppy controller (Western Digital FD1793, a TMS5501 console UART, and the RDOS boot PROM in one card) boots **CDOS**, and the **Dazzler** paints S-100's first color graphics out of a framebuffer in main RAM, with the **D+7A** analog/parallel card and a **JS-1 joystick** (real gamepad or keyboard) to drive games on it. **SD Systems** shows up as a matching trio — the **SBC-100/200** single-board Z80 (an 8251 console that auto-bauds to your terminal, and later a full interrupt-driven CP/M boot with the onboard PROM switching itself out), the **VersaFloppy** WD177x controller booting SDOS, and the **VDB-8024** 80×24 video terminal card. The **Tarbell #1011** and **#2022** floppy controllers boot CP/M entirely on their own, no monitor involved, straight off their own boot PROM — and the **#2022** now boots a disk whose CBIOS moves sectors by **DMA**, driving the card's on-board Intel 8257 to steal S-100 bus cycles and drop each sector into memory itself, the first board in the simulator to master the bus (`altairsim tarbelldd-dma.toml`). The **iCOM FD3712/FD3812** 8″ floppy controller — a programmed-I/O command engine rather than a WD177x card, with its driver in a boot PROM up in high memory — boots CP/M 2.2 in both single density (FD3712) and double density (FD3812), and both revisions of iCOM's own **FDOS** disk operating system — the original **FDOS-I** and the later **FDOS-III** — each off its own PROM (`altairsim icom`). And the reproduction era shows up too: **S100Computers**' modern boards boot **CP/M 3** off flash, the **V2 Z80 CPU board**'s MASTER monitor EEPROM loading it with an `I` command off the new **Dual SD** controller — two microSD cards on the bus as raw 512-byte-sector drives, each card one CP/M drive stored as a truncated `.img` beside a `.geo` sidecar that declares the full card (so a card that is hundreds of megabytes on real flash ships as a couple of megabytes), with the **Console I/O** board as its Propeller-based serial console (`altairsim dualsd`). Its **IDE-AB** companion does the same off **CompactFlash** — an 8255 driving a CF card's IDE bus, booted with the monitor's `P` command as drives A:/B: — and because both halves lay a card out identically, one system image is interchangeable between them; the two together make the full combination board, CompactFlash as A:/B: and microSD as C:/D:, a single CP/M 3 spanning all four drives (`altairsim`

`dualide`, `altairsim dualidesd`). On the MITS side, the **88-HDSK** boots CP/M off a hard disk, **88-PIO/88-4PIO** add parallel I/O, **88-LPC** drives a real line printer, **88-UIO** combines a serial port and a cassette deck on one card, and the **8800b Turnkey Module** brings up a front-panel-less Altair with its boot PROM jammed onto the bus at RESET. Two boards round it out: the **PMMI MM-103**, the first S-100 modem (dial and answer a real phone line over TCP), and **usio**, a serial card you describe by strap instead of one we picked, with built-in profiles for the Cromemco TU-ART, IMSAI SIO-2 and CompuPro serial channels. A new built-in ROM, **ROM BASIC**, boots Altair BASIC 4.1 straight out of PROM with the full 48K free underneath it.

A third validated CPU core: the Intel 8085

The 8080 and Z80 are joined by a documented **Intel 8085** core — the 8080's binary superset, with `RIM` / `SIM` and the on-chip interrupt system (the non-maskable `TRAP` and the maskable `RST 5.5/6.5/7.5`, each with its mask and pending latch, in hardware priority order) that the 8080 never had. It is faithful where the two chips actually differ: the 8085's logical `AND` always sets the auxiliary-carry flag, where the 8080 derives it from the operands. Like the other two cores it earns its place at the gate rather than by inspection — Ian Bartholomew's **8085** instruction-set exerciser, whose expected CRCs were read off real 8085 silicon, runs its 2.9 billion instructions against the core on every CI push, on all three platforms.

The core is now something you can drive: an **`8085` machine** (`altairsim 8085` — an 8085, 64K, and a 2SIO console, the direct analog of the `z80` machine) and an **8085 disassembler and assembler**, so `DISASM` and `EDIT` speak the 8085 where that core is active. `RIM` and `SIM` decode as themselves; the ten *undocumented* 8085 opcodes are named and flagged the way `DDT` flags a byte outside the published set (`??= 08  *DSUB`), since the core still runs those slots as `NOP`. Their faithful behavior and the V/K flag bits remain the open work of the 8085.

Disks and tapes the guest can build, not just read

A hard-sector disk or a cassette tape can now be created **blank** and brought to life by the guest's own software: `MOUNT … CREATE` writes an empty image, and it grows one sector — or one byte of tape — at a time as the guest's FORMAT program defines it, exactly as unformatted media behaves on real hardware. Soft-sector disks caught up too: the Tarbell and VersaFloppy controllers now honor the WD177x Write Track command, so CP/M's `FORMAT` and SDOS's disk formatter lay down a bootable disk from nothing, across every geometry those cards support. `MOUNT` also now reads **ImageDisk (`.IMD`)** files directly, converting the interleaved container to the raw sector-linear image the disk boards expect. On the cassette side, both cassette decks show where the head is (`mm:ss / total (percent)`), a new `WIND` command moves it to a time so a multi-program tape is finally navigable, a `stop` mark parks playback at a boundary, and `EXTRACT` splits a multi-program recording into one file per program. Tapes the simulator writes now load on a real Sol-20, because the cassette modem's output is modeled the way the real hardware's clock-divider and filter actually shape a signal, not as a clean oscillator a real deck could never produce.

A debugger with sharper eyes

Cycle breakpoints (`BREAK MEM` / `BREAK IO`) now stop **before** the triggering instruction runs instead of after, so a breakpoint on a port read shows you the registers exactly as they were the instant the instruction fired — no port read, no byte written, nothing to unwind. Cycle breakpoints now take a condition as well: `BREAK MEM W 100 IF B==0` or `BREAK IO R 10 IF C==1` stops only on the access whose registers hold, and `BREAK IO R 10 LOADS A>7F` tests the byte an `IN` actually read rather than the state it read it with. `BREAK TAPE STOP` adds a third kind of breakpoint, alongside PC and bus-cycle stops, that halts the instant a cassette reaches its own auto-stop. `HISTORY` now defaults to a flight recorder of CPU instructions rather than raw bus cycles, and its bus view (`HISTORY BUS`) names which board drove and which answered every cycle it logs. `EDIT`, the byte-at-a-time memory editor, now assembles a full instruction where a byte would go. Loaded symbols make disassembly and single-stepping read by name instead of address, `SAVE` can write a disassembly or octal listing to a file, and the monitor can display and parse addresses, ports and bytes in authentic split **octal**, MITS's own notation. A new diagnostic-channel facility (`SET <id> DEBUG=`, `SHOW DEBUG`) lets individual boards and shared chips narrate what they're doing — a disk stepping heads, a UART taking a byte — each line stamped with the PC that caused it, aimed at the console, a file, or a `log` that tees your whole session transcript to disk. `SET BUS UNCLAIMED` catches a guest reaching for a board that was never fitted, and a bare `!` drops you to your host shell without losing the machine's state underneath it.

A prompt that helps you type

`Tab` now completes commands, board ids, unit names, property names and their legal values at the monitor prompt, reading the candidates straight off the running machine so a board you plug in is completable immediately. The line editor grew word motion, `Home` / `End`, and kill-to-end-of-line, and command history now survives a restart, saved per project directory. Asking what you can build is one family of commands now — `SHOW BOARDS`, `SHOW BOARD <type>`, `SHOW MACHINES`, `SHOW MACHINE <name>` — with legal values listed under every property. A machine file can leave you a note: a `#>` comment line prints itself to the operator when the machine loads, the one or two sentences a config's author needs you to see before you start typing. And `HELP` now answers at whatever level you actually asked for, instead of always dropping you at the top.

The video window and the joysticks behind it

A video window now opens locked to its picture's aspect ratio and resizes proportionally from the first drag, with an even bezel on all four sides and its title bar reading "simulator stopped" whenever the guest is halted. How big it opens is a per-board `width` in pixels, so each card sizes itself off its own resolution; the Dazzler's tiny frame gets its own auto-scaling so it lands near the same size as a VDM-1's instead of a sixth of it. The D+7A now reports what's actually behind each joystick console — a named gamepad, the keyboard, or nothing — and a new `SHOW JOYSTICKS` lists every controller the host can see; two consoles

default to two different gamepads automatically, so a pair of controllers just works with no configuration at all.

A terminal in its own window

A serial line can now open its own **built-in terminal window** the simulator draws itself — `CONNECT sio0:a terminal`, or `connect = "terminal"` in a machine file — so the machine's console and the monitor's command prompt live in separate windows with no telnet client, no external emulator, and nothing to install on any platform. It speaks four dialects the way period software expects them, `? emulation=` picking one: **VT100/ANSI** (the default, enough of it to run a full-screen editor), the dumb CP/M **ADM-3A**, the **VT52**, and the Heath/Zenith **H19** — the last three being terminals no modern emulator provides, which was the whole point. `?size=COLSxROWS` sets the geometry (80×24 by default). Every serial board gets it for free, and in a headless build the endpoint simply refuses cleanly at CONNECT rather than opening a line nobody can see.

The built-in terminal now carries the same fold-the-bytes settings the console has, in a `[terminal]` section (`SET TERMINAL`, `SHOW TERMINAL`, or a block in a machine file): `strip7out`, `strip7in`, `upper`, `echo`, `bell`, `bsdel`, and a `cr = cr | crlf` line-ending option. They earn their keep on a period **even-parity monitor** — the MITS Programming System II computes parity into bit 7, so it sends a carriage return as `8D`, which a raw terminal printed as a glyph instead of homing the cursor, feeding every line without a return; `strip7out` masks it, exactly as it does for MITS BASIC on the console. And the run banner no longer claims `(no console connected)` when the console is live on a `terminal:` window or a `socket:` — it now names the line (`(console on terminal)`), keeping the old phrase only for a machine with no serial line up at all.

Save a machine, and pick it back up later

`SNAPSHOT <file>` writes the whole machine's state — every board's registers and RAM, the clock, and the CPU down to the microstate a register dump never shows — to a small, checksummed file, and `RESTORE <file>` loads it back into a machine of the same shape, refusing a corrupt or mismatched file before it touches a single byte of the machine you're running.

Building, driving and reading altairsim got easier

A clean clone now goes from nothing to a running binary with one command, `build.sh` or `build.bat`, with SDL3 staying entirely optional. Every warning the compiler can find now fails continuous integration outright, so nothing that used to be quiet creeps back in unnoticed, and a local sanitizer build is one flag away when a bug needs one. The AI-driving guide gained the step it was missing — how to actually register altairsim as an MCP server — plus a worked example where an assistant assembles a small buggy program, single-steps it to find the fault, and fixes it, entirely over the wire. And a font fix means copying a line out of any built PDF — the manual, an example's README, this changelog — now pastes back as the words it actually says, not text with spaces jammed into the middle of them.

0.3.0

0.3.0 adds no machines and no boards. It changes one thing, and it is the thing the manual has described all along: the copy you download now opens the video window.

The window the manual documents is finally in the package

Every release through 0.2.0 shipped a **headless** binary. SDL3 was not compiled into it, so the VDM-1 and Sol-20 windows the boards and configuring chapters describe at length could not open from anything you were handed — the machines ran, and drew nothing. `SHOW VERSION` said so in a row nobody had reason to read:

```
altairsim> SHOW VERSION
  altairsim  0.3.0
  video      SDL3 -- windowed        <- 0.2.0 and before, the download read: none -- headless
  commit     v0.3.0
  tree       clean
```

0.3.0 is the first release whose downloaded package reads `SDL3 -- windowed`. Run `altairsim sol20`, and a window opens. That is the headline, and most of the rest of this entry is how it was made true on every platform at once.

Nothing to install, on any of the four

The packages are built on the hardware they target now, rather than assembled by CI, and SDL3 is **linked statically into the binary**. So there is no `SDL3.dll` to sit beside the `.exe`, no `.framework`, no `libSDL3.so.0`, and nothing to install before the program runs — the claim altairsim has always made for itself is now true of its video too. On Windows the C runtime is static as well, so a clean machine that has never had a compiler on it runs the `.exe` with no Microsoft redistributable to chase.

The one visible cost is a single extra file in the archive — `LICENSE-SDL3`, SDL3's zlib licence, because SDL3's code now travels inside the binary and its licence travels with it.

macOS ships as two builds, each tested on its own hardware

0.2.0 shipped one *universal* macOS archive. 0.3.0 ships two — `altairsim-0.3.0-macos-arm64` and `altairsim-0.3.0-macos-x86_64` — and the reason is honesty, not size. The universal binary's Intel half had been built and tested by nobody, because the machine that produced it was Apple Silicon; the previous two releases said so in their own notes. Each of the two archives is now built **and** run on the

architecture it targets, so the Intel download is exercised on an Intel Mac before it ships. Take the one that matches your Mac; `altairsim --version` and the download filename both name the architecture.

Windows, proven end to end

The Windows package is built with MSVC, links SDL3 and the C runtime statically, passes the full test suite on the machine that builds it, and opens a real VDM-1 window that a person has sat in front of. `dumpbin /dependents` on the shipped `.exe` shows system DLLs only — no `SDL3.dll`, no `VCRUNTIME140`. It is held to the same bar as the other three, and it is no longer an asterisk on the release.

The four downloads

Platform	File
macOS Apple Silicon	`altairsim-0.3.0-macos-arm64.tar.gz`
macOS Intel	`altairsim-0.3.0-macos-x86_64.tar.gz`
Linux x86_64	`altairsim-0.3.0-linux-x86_64.tar.gz`
Windows x86_64	`altairsim-0.3.0-windows-x86_64.zip`

Each holds the program, this changelog, the **User Manual**, `DRIVING-WITH-AI.md`, both licences, and `examples/` — four machines that boot, media included. Unzip it and run it; nothing needs fetching first.

What did not change

The simulator itself is byte-for-byte the machine 0.2.0 was — the same machines, the same boards, the same two CPU cores. The holes named in the manual's introduction are still holes: no snapshot, no replay, the six reserved monitor verbs still reserved, still no audio. Everything that moved is in the box the program arrives in.

0.2.0

The second release. It added machines and made the video window behave like a window — but note that the packages were still **headless** (see 0.3.0): everything below was true of a build made *with* SDL3, which is not what the archives carried until 0.3.0.

Three more monitors that boot from a bare command line

`amon`, `acuter` and `cdbl` became machines, so Martin Eberhard's Altair ROMs boot by name — nothing to fetch, nothing to mount:

```
altairsim amon          AMON 3.1 in a 4K EPROM at F000 -- a full-featured Altair monitor
altairsim acuter        ACUTER at F000 -- CUTER on a plain Altair, driving a terminal
altairsim cdbl          the default machine, with the Combo Disk Boot Loader in the socket
```

The ROM images shipped in 0.1.0 already; what was new is that each got a machine built around it — the whole distance between shipping an image and being able to run it. `hdbl` was deliberately left out: it boots an 88-HDSK hard disk, and there is no 88-HDSK board here.

The video window behaves like a window

- **It does not steal the keyboard when it opens.** The terminal keeps the keys while you type at the monitor prompt; `SET DISPLAY focus=on` hands them to the guest, stopping it hands them back.
- **It is named after the machine**, so `sol20` and `vdm1` are two windows you can tell apart.
- **It is sized to fit the screen it opened on**, and **arrows and HOME reach the guest**.
- **Closing it stops the guest**, instead of leaving a machine running with nothing to draw on.

Two of those were bugs worth naming: typed input could lag a whole frame, and the VDM-1 could repaint hundreds of times per emulated millisecond. Both gone. The VDM-1's cursor now blinks on the board's own oscillator — wall-clock time, as the hardware did.

Disk BASIC, and smaller things

- `examples/diskbasic` boots **Altair Disk BASIC 4.1** off a floppy, media included — a fourth worked example alongside CP/M, cassette BASIC and the Sol-20.
- A binary now names the commit it was built from: `SHOW VERSION` and `--version` carry it, so a report against a nightly or a CI artifact can be traced to the code that produced it.
- `writeprotect` is accepted wherever `readonly` is, in machine files and at `SET`.
- The manual and the program say **board**, not card.

0.1.0

The first release — a simulator for the MITS Altair 8800 and the S-100 bus, in C++20, with **nothing to fetch**: the TOML parser, the JSON encoder and the line editor are all in-tree, so a fresh clone builds with a C++20 compiler and CMake and no network.

Two validated CPU cores

Both cleared their gate before a single board was built on them, and for the release the exercisers were re-run on all three platforms — roughly 15 billion instructions each:

- **8080** — TST8080, 8080PRE, CPUTEST, 8080EXM
- **Z80** — ZEXDOC, ZEXALL

Boards and machines

88-2SIO · 88-SIO · 88-ACR · 88-DCDD · 88-MDS · 88-VI/RTC · 88-C700 · front panel · memory · 8080 CPU · Z80 CPU · VDM-1 · Processor Technology Sol-PC · Host Bridge (our own design).

CP/M 2.2 cold-boots from both 8-inch and 5.25-inch disks. MITS 4K and 8K BASIC and Programming System II load from cassette — including **real `.WAV` audio, played and recorded**, at measured CUTS/ACR parameters.

Debugging, and an assistant that can drive it

Breakpoints, watchpoints, tracepoints, conditional breaks (`BREAK … IF`), execution history, and symbolic reference loaded from `.PRN` and `.SYM` files — DDT and SID run under it, self-modifying RST 7 breakpoints and all. An **MCP server** lets an AI assistant drive a running guest.